

《数据库系统原理》报告

数据库系统原理作业 3

张鲋泮

北京交通大学计算机科学与技术学院 AI2201 班 22281052@bjtu.edu.cn

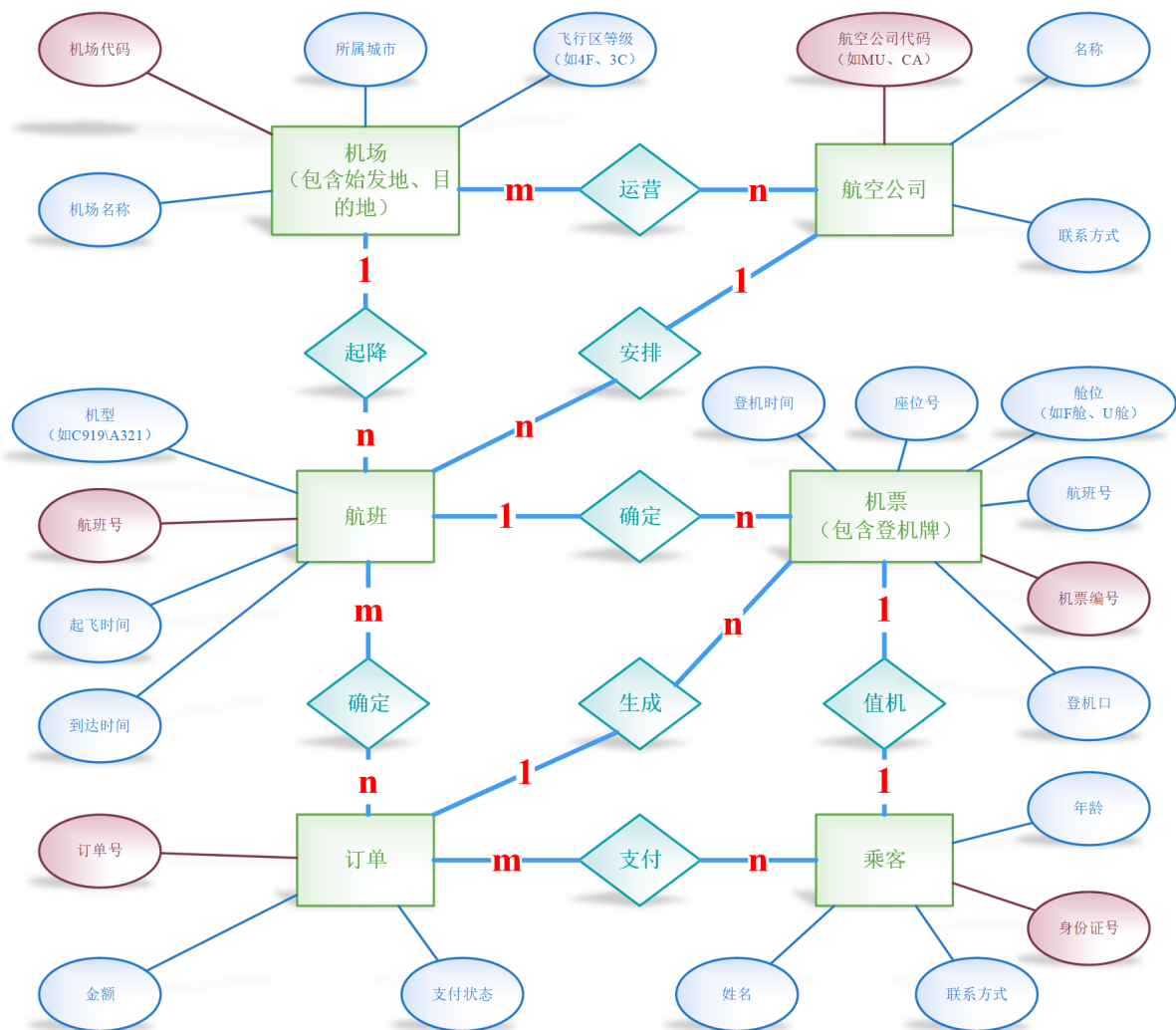


图 1: 航空公司航班查询业务模型 E-R 图, 来自于作业 2

请将你作业 2 中设计的模式转化成数据库中的表, 并完成以下任务:

1 题目 1

题目要求: 按照下表格式, 将实验中描述所有涉及到的表的结构

表 1: Faculty 表结构

表名	Faculty				
数据库用户	root				
主键	ID				
外键	RolesID				
排序字段	ID				
索引字段	RolesID				
字段名称	数据类型	允许为空	唯一	默认值	约束条件
ID	Int	N	Y		主键 外键
IDCode	Varchar	N	N		
Password	Varchar	N	N		
RolesID	Varchar	N	N		
Name	Varchar	N	N		
EngName	Varchar	N	N		
Gender	tinyint	N	N		

其他附加说明：

- 性别 (*Gender*) 的值域范围为 $(0,1)$ ，其中：

1. 0 表示女性
2. 1 表示男性

-

针对题目 1，我将作业 2 中设计的六个实体类型，转化成数据库中的表结构描述。

表 2: Airport 表结构

表名	Airport				
数据库用户	root				
主键	airport_code				
外键	无				
排序字段	airport_code				
索引字段	city				
字段名称	数据类型	允许为空	唯一	默认值	约束条件
airport_code	varchar(10)	N	Y		主键
airport_name	varchar(100)	N	N		
city	varchar(50)	N	N		
flight_zone	varchar(10)	N	N		

其他附加说明：

- 机场代码 (airport_code) 是该表的主键，唯一且不可为空。
- 机场名称 (airport_name) 和所属城市 (city) 字段不能为空。
- 飞行区等级 (flight_zone) 是可选字段，可以为空，默认值为 NULL。

表 3: Airline 表结构

表名	Airline				
数据库用户	root				
主键	airline_code				
外键	无				
排序字段	airline_code				
索引字段	airline_name				
字段名称	数据类型	允许为空	唯一	默认值	约束条件
airline_code	varchar(10)	N	Y		主键
airline_name	varchar(100)	N	N		
contact_info	varchar(100)	Y	N	NULL	

其他附加说明：

- 航空公司代码 (airline_code) 是该表的主键，唯一且不可为空。
- 联系方式 (contact_info) 是可选字段，可以为空，默认值为 NULL。

表 4: Flight 表结构

表名	Flight				
数据库用户	root				
主键	flight_number				
外键	departure_airport, arrival_airport, airline_code				
排序字段	departure_time				
索引字段	airline_code				
字段名称	数据类型	允许为空	唯一	默认值	约束条件
flight_number	varchar(20)	N	Y		主键
air_model	varchar(50)	N	N		
departure_time	datetime	N	N		
arrival_time	datetime	N	N		
departure_airport	varchar(10)	N	N		外键 → Airport
arrival_airport	varchar(10)	N	N		外键 → Airport
airline_code	varchar(10)	N	N		外键 → Airline

其他附加说明：

- 航班号（flight_number）是该表的主键，唯一且不可为空。
- 出发机场（departure_airport）和到达机场（arrival_airport）是外键，分别关联到 Airport 表的 airport_code 字段。
- 航空公司（airline_code）是外键，关联到 Airline 表的 airline_code 字段。
- 起飞时间（departure_time）和到达时间（arrival_time）字段不可为空，且代表航班的起降时间。

表 5: Ticket 表结构

表名	Ticket				
数据库用户	root				
主键	ticket_id				
外键	flight_number, id_card				
排序字段	boarding_time				
索引字段	seat_number				
字段名称	数据类型	允许为空	唯一	默认值	约束条件
ticket_id	varchar(20)	N	Y		主键 外键 → Flight 外键 → Passenger
flight_number	varchar(20)	N	N		
id_card	varchar(20)	N	N		
seat_number	varchar(10)	N	N		
gate	varchar(10)	Y	N	NULL	
boarding_time	datetime	N	N		
cabin_seat	varchar(10)	Y	N	NULL	

其他附加说明：

- 机票编号（ticket_id）是该表的主键，唯一且不可为空。
- 航班号（flight_number）是外键，关联到 Flight 表的 flight_number 字段。
- 身份证号（id_card）是外键，关联到 Passenger 表的 id_card 字段。
- 座位号（seat_number）和登机口（gate）是可选字段，可以为空，默认值为 NULL。
- 登机时间（boarding_time）是航班的实际登机时间，必须填写。
- 舱位（cabin_seat）是可选字段，可以为空，默认值为 NULL。

表 6: Orderinfo 表结构

表名	Orderinfo				
数据库用户	root				
主键	order_id				
外键	id_card				
排序字段	order_id				
索引字段	payment_status				
字段名称	数据类型	允许为空	唯一	默认值	约束条件
order_id	varchar(20)	N	Y		主键
id_card	varchar(20)	N	N		外键 → Passenger
moneys	decimal(10,2)	N	N		
payment	INT	N	N	0	CHECK (payment IN (0, 1))

其他附加说明：

- 订单号（order_id）是该表的主键，唯一且不可为空。
- 身份证号（id_card）是外键，关联到 Passenger 表的 id_card 字段。
- 总金额（moneys）字段记录订单的总费用，不能为空。
- 支付状态（payment）字段，默认值为“0”为“Unpaid”，“1”为“Paid”，并且只能是 0 或 1 其中之一，使用 CHECK 约束进行限制。

表 7: Passenger 表结构

表名	Passenger				
数据库用户	root				
主键	id_card				
外键	无				
排序字段	name				
索引字段	age				
字段名称	数据类型	允许为空	唯一	默认值	约束条件
id_card	varchar(20)	N	Y		主键
passname	varchar(50)	N	N		
age	int	N	N		CHECK (age >= 0 AND age < 120)
phone	varchar(100)	Y	N	NULL	

其他附加说明：

- 身份证号（id_card）是该表的主键，唯一且不可为空。

- 姓名 (passname) 和年龄 (age) 字段不能为空。
- 年龄 (age) 字段使用 CHECK 约束, 限制其范围为 0 到 120 岁之间。
- 联系方式 (phone) 是可选字段, 可以为空, 默认值为 NULL。

2 题目 2

题目要求: 根据以上定义, 写出各表的建表语句, 并在你选的关系型数据库平台上建立各个表, 请将建表语句统一写在扩展名为 *sql* 的文件中, 构建一个建库脚本文本。

由关系可以编写语句。因此, 我将以下代码, 写入到 .sql 文件内。

```

1  -- 创建数据库
2  CREATE DATABASE airlinedb;
3  USE airlinedb;
4  -- 机场表
5  CREATE TABLE Airport (
6      airport_code VARCHAR(10) PRIMARY KEY,
7      airport_name VARCHAR(100) NOT NULL,
8      city VARCHAR(50) NOT NULL,
9      flight_zone VARCHAR(10) NOT NULL
10 );
11 -- 航空公司表
12 CREATE Table Airline (
13     airline_code VARCHAR(10) PRIMARY KEY,
14     airline_name VARCHAR(100) NOT NULL,
15     contact_info VARCHAR(100)
16 );
17 -- 航班表
18 CREATE Table Flight (
19     flight_number VARCHAR(20) PRIMARY KEY,
20     air_model VARCHAR(50) NOT NULL,
21     departure_time DATETIME NOT NULL,
22     arrival_time DATETIME NOT NULL,
23     departure_airport VARCHAR(10) NOT NULL,
24     arrival_airport VARCHAR(10) NOT NULL,
25     airline_code VARCHAR(10) NOT NULL,
26     Foreign Key (departure_airport) REFERENCES Airport (airport_code)
27     ,
28     Foreign Key (arrival_airport) REFERENCES Airport (airport_code),
29     Foreign Key (airline_code) REFERENCES Airline (airline_code)
30 );
31 -- 乘客表

```

```

31 CREATE Table Passenger (
32     id_card VARCHAR(20) PRIMARY KEY,
33     passname VARCHAR(50) NOT NULL,
34     age INT NOT NULL CHECK (
35         age >= 0
36         AND age < 120
37     ),
38     phone VARCHAR(100)
39 );
40 -- 机票表
41 CREATE Table Ticket (
42     ticket_id INT AUTO_INCREMENT PRIMARY KEY,
43     flight_number VARCHAR(20) NOT NULL,
44     id_card VARCHAR(20) NOT NULL,
45     seat_number VARCHAR(10) NOT NULL,
46     gate VARCHAR(10),
47     boarding_time DATETIME NOT NULL,
48     cabin_seat VARCHAR(10),
49     Foreign Key (flight_number) REFERENCES Flight (flight_number),
50     Foreign Key (id_card) REFERENCES Passenger (id_card)
51 );
52 -- 订单表
53 CREATE Table Orderinfo (
54     order_id INT AUTO_INCREMENT PRIMARY KEY,
55     id_card VARCHAR(20) NOT NULL,
56     moneys DECIMAL(10, 2) NOT NULL,
57     payment INT NOT NULL CHECK (payment in (0, 1)),
58     Foreign Key (id_card) REFERENCES Passenger (id_card)
59 );

```

之后，将以上 sql 文件导入到 Mysql 系统内。登录 MySQL Command Line Client 后，输入以下语句，建库语句成功导入（如图2）：

```

1 source D:\Learning_ComputerLanguages\learning_language_sql\1_Database
  \1_exp3\exp3.sql

```

```

mysql> source D:\Learning_ComputerLanguages\learning_language_sql\exp3.sql
Query OK, 0 rows affected (0.00 sec)

Query OK, 1 row affected (0.01 sec)

Database changed
Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.04 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.04 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.06 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.03 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.06 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.05 sec)

```

图 2: 建库语句成功导入

3 题目 3

题目要求：掌握选用的关系型数据库管理系统的控制台插入数据的不同方法（执行数据批量插入脚本、窗口界面表格式手工录入、命令行交互式录入），实际填入测试数据，以验证你所设计的数据模型的合理性和完整性，注意验证三种完整性约束。

3.1 不同方法插入测试数据

由题目可知，插入数据的常用方法有 3 种：**执行数据批量插入脚本、窗口界面表格式手工录入、命令行交互式录入**。首先先编写了测试数据如下：

```

1 INSERT INTO Airport VALUES
2 ('PEK', 'Beijing_Capital_International_Airport', 'Beijing', '4F'),
3 ('PVG', 'Shanghai_Pudong_International_Airport', 'Shanghai', '4F');
4
5 INSERT INTO Airline VALUES
6 ('CA', 'Air_China', '010-95583'),
7 ('MU', 'China_Eastern', '021-95530');
8
9 INSERT INTO Flight VALUES
10 ('CA1234', 'Boeing737', '2024-04-10_08:00:00', '2024-04-10_10:30:00',
11    'PEK', 'PVG', 'CA'),
12 ('MU5678', 'AirbusA320', '2024-04-10_09:00:00', '2024-04-10_11:45:00',
    'PVG', 'PEK', 'MU');

```



```

13 INSERT INTO Passenger VALUES
14 ('110101199001011234', 'Zhangsan', 30, '13800138000'),
15 ('110101199002022345', 'Lisi', 28, '13900139000');
16
17 -- C为商务舱, Y为经济舱
18 INSERT INTO Ticket (flight_number, id_card, seat_number, gate,
19     boarding_time, cabin_seat) VALUES
20 ('CA1234', '110101199001011234', '12A', 'B12', '2024-04-10 07:20:00',
21     'Y'),
22 ('MU5678', '110101199002022345', '14C', 'A05', '2024-04-10 08:30:00',
23     'C');
24
25 INSERT INTO Orderinfo (id_card, moneys, payment) VALUES
26 ('110101199001011234', 1250.50, 1),
27 ('110101199002022345', 3150.00, 0);

```

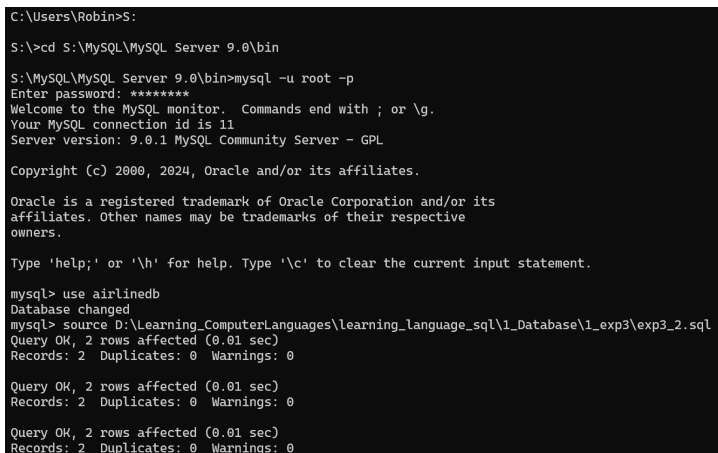
3.1.1 方法一：执行数据批量插入脚本

此方法通过.sql 文件将多条数据一次性导入数据库。首先进入命令行窗口，cd 进入到 MySQL 安装目录的 bin 文件下。输入 “mysql -u root -p”，再输入数据库密码进入到数据库界面。之后进入 airlinedb 数据库，执行 source 语句（如图3）。（类似于登录 MySQL Command Line Client 后执行 source 语句）cmd 窗口执行语句如下：

```

1 S:
2 cd S:\MySQL\MySQL Server 9.0\bin
3 mysql -u root -p
4 use airlinedb
5 source D:\Learning_ComputerLanguages\learning_language_sql\1_Database
   \1_exp3\exp3_2.sql

```



```

C:\Users\Robin>S:
S:\>cd S:\MySQL\MySQL Server 9.0\bin
S:\MySQL\MySQL Server 9.0\bin>mysql -u root -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 9.0.1 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use airlinedb
Database changed
mysql> source D:\Learning_ComputerLanguages\learning_language_sql\1_Database\1_exp3\exp3_2.sql
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

```

图 3: cmd 窗口插入数据

3.1.2 方法二：窗口界面表格式手工录入

此方法使用 Navicat 等数据库可视化工具直接在表格界面手动输入数据。本人在作业 1 时，下载了 Navicat Premium 17，因此此部分可以运用 Navicat Premium 17 导入数据 (如图4)。由于手工录入较为麻烦，因此此处忽略这种方式。

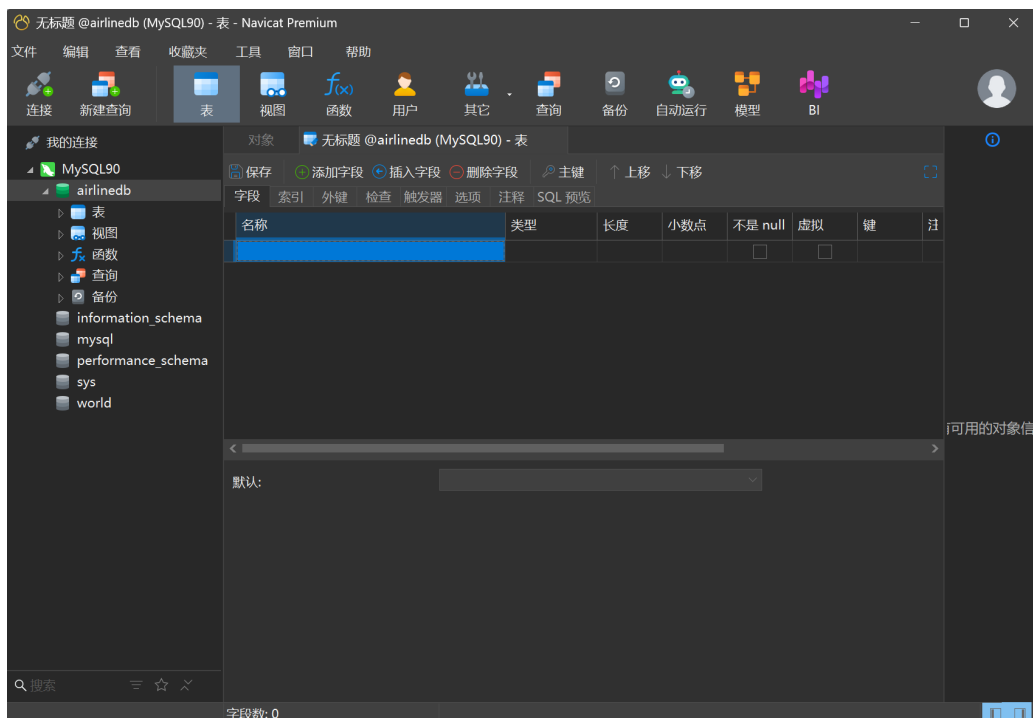
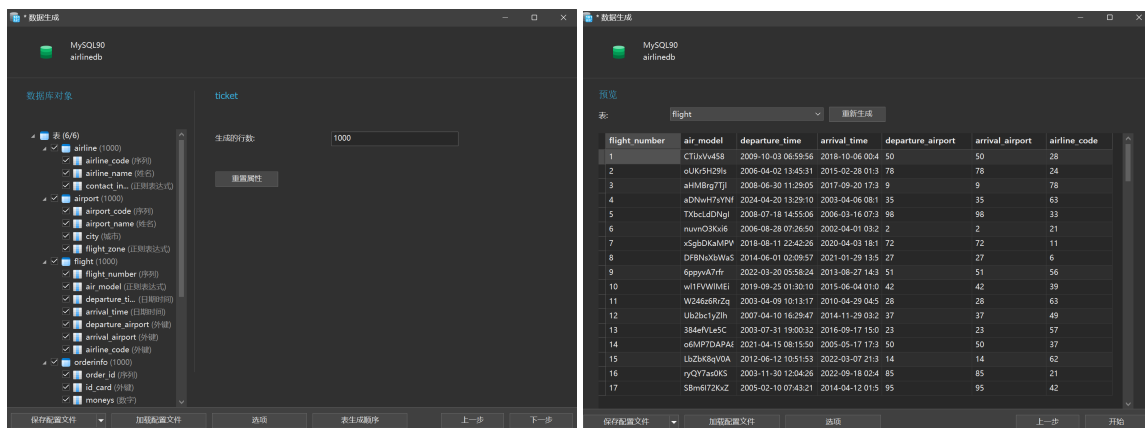


图 4: Navicat Premium 17 手工录入数据

但是，Navicat Premium 17 提供了生成数据的方式。因此可以在测试的时候使用此方式 (如图5a)。但是测试数据无法符合实际要求，得需要进行调整 (如图5b)。



(a) Navicat Premium 17 生成数据 1

(b) Navicat Premium 17 生成数据 2

图 5: Navicat Premium 17 生成数据

3.1.3 方法三：命令行交互式录入

此方法使用 MySQL Command Line Client，通过逐条输入 SQL 命令交互式插入数据。打开 MySQL Command Line Client 并登录后，输入语句即可 (如图6)。

```

-> ('CA1234', 'Boeing737', '2024-04-10 08:00:00', '2024-04-10 10:30:00', 'PEK', 'PVG', 'CA'),
-> ('MU5678', 'AirbusA320', '2024-04-10 09:00:00', '2024-04-10 11:45:00', 'PVG', 'PEK', 'MU');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Passenger VALUES
-> ('110101199001011234', 'Zhangsan', 30, '13800138000'),
-> ('110101199002022345', 'Lisi', 28, '13900139000');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> -- C为商务舱, Y为经济舱
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO Ticket (flight_number, id_card, seat_number, gate, boarding_time, cabin_seat) VALUES
-> ('CA1234', '110101199001011234', '12A', 'B12', '2024-04-10 07:20:00', 'Y'),
-> ('MU5678', '110101199002022345', '14C', 'A05', '2024-04-10 08:30:00', 'C');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Orderinfo (id_card, moneys, payment) VALUES
-> ('110101199001011234', 1250.50, 1),
-> ('110101199002022345', 3150.00, 0);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>

```

图 6: 命令行交互式录入

3.2 验证三种完整性约束

3.2.1 实体完整性

实体完整性要求主键字段必须唯一，且不能为空。测试语句如下：

```

1 -- 插入一个新机场
2 -- 预期结果插入正常
3 INSERT INTO Airport VALUES ('CAN', 'Guangzhou_Baiyun_International_
   Airport', 'Guangzhou', '4F');
4
5 -- 尝试插入与已有主键重复的机场
6 -- 预期结果 ERROR 1062 (23000): Duplicate entry 'PEK' for key '
   PRIMARY'
7 INSERT INTO Airport VALUES ('PEK', 'Some_Airport', 'AnyCity', '4F');

```

测试结果如图7，与预期相符。

```

mysql> source D:\Learning_ComputerLanguages\learning_language_sql\1_Database\1_exp3\exp3_3.sql
Query OK, 0 rows affected (0.00 sec)

Query OK, 1 row affected (0.01 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

ERROR 1062 (23000): Duplicate entry 'PEK' for key 'airport.PRIMARY'

```

图 7: 实体完整性

3.2.2 参照完整性

参照完整性要求子表中的外键值必须在其对应的父表中已存在。测试语句如下：

```
1 -- 以 CA、PEK、PVG 为外键的航班
2 -- 预期结果插入正常
3 INSERT INTO Flight VALUES ('CA9999', 'AirbusA330', '2024-04-12_
   10:00:00', '2024-04-12_13:00:00', 'PEK', 'PVG', 'CA');
4
5 -- 插入航班，airline_code 不存在
6 -- 预期结果 ERROR 1452 (23000): Cannot add or update a child row: a
   foreign key constraint fails
7 INSERT INTO Flight VALUES ('AB1234', 'Boeing777', '2024-05-01_
   10:00:00', '2024-05-01_12:00:00', 'PEK', 'PVG', 'AB');
```

测试结果如图8，与预期相符。

```
Query OK, 1 row affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.00 sec)
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails ('airlinedb`.`flight`, CONSTRAINT `flight_ibfk_3` FOREIGN KEY (`airline_code`) REFERENCES `airline` (`airline_code`))
```

图 8: 参照完整性

3.2.3 用户定义完整性

用户定义完整性要求字段值必须满足人为定义的规则（如年龄区间、支付状态只能是 0 或 1）。测试语句如下：

```
1 -- 合法年龄
2 -- 预期结果插入正常
3 INSERT INTO Passenger VALUES ('110101199003035678', 'Wangwu', 25, '
   13700000000');
4 -- 合法支付状态
5 -- 预期结果插入正常
6 INSERT INTO Orderinfo (id_card, moneys, payment) VALUES ('
   110101199003035678', 999.99, 1);
7
8 -- 非法支付状态（只能是0或1）
9 -- 预期结果 Check constraint 'Orderinfo_chk_1' is violated
10 INSERT INTO Orderinfo (id_card, moneys, payment) VALUES ('
   110101199001011234', 88.88, 2);
11 -- 非法年龄（超过上限）
12 -- 预期结果 Check constraint 'Passenger_chk_1' is violated
13 INSERT INTO Passenger VALUES ('110101199004046789', 'Zhaoliu', 150, '
   13600000000');
```

测试结果如图9，与预期相符。

```
Query OK, 1 row affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.00 sec)
ERROR 3819 (HY000): Check constraint 'orderinfo_chk_1' is violated.
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.00 sec)
ERROR 3819 (HY000): Check constraint 'passenger_chk_1' is violated.
```

图 9: 用户定义完整性

4 题目 4

题目要求：请根据以上设计结果，重新完善与整理作业 2 中所设计的模式
目前来看，暂时不需要进行相关操作。因此暂不进行完善作业 2 中所设计的模式。
原作业 2 中所设计的模式如下（如图10与图11）。



图 10: UI 主要界面



图 11: UI 次要界面

5 题目 5

题目要求：请设计单表查询、多表连接查询语句，查询表中的内容，并截图证明在进行本题之前，本人扩充了测试数据（详见文件 exp3_adddata.sql），以更好展示梳理数据结果。

5.1 单表查询

```

1  -- 查询 all 乘客信息
2  SELECT * FROM Passenger;
3  -- 查询所有已经支付的订单
4  SELECT * FROM Orderinfo WHERE payment=1;
5  -- 查询 CA1234 票务信息
6  SELECT id_card, gate, boarding_time FROM Ticket WHERE flight_number='
    MU4241';

```

查询的结果如图12所示。

Query OK, 0 rows affected (0.00 sec)

id_card	passname	age	phone
110101195010215457	Robin	42	2130427306
110101195011276882	Rachel	50	(695)853-1407x0976
110101195005186187	Mary	70	572.245.9752x801
110101195510198228	Kamberly	28	639.453.5941x8088
110101195510276691	Sheryl	68	042.636.5506
110101195603061279	Patrick	42	(746)521-7149x5387
110101195710268104	Matthew	57	3250111485
110101195902078356	Angellca	39	671.252.2284x988
11010119600204870	Candice	59	+1-865-272-5344x8720
110101196010073893	Amber	38	724.764.6974x6847
110101196205083417	Jillian	62	001-037-770-3037x77578
110101196206102759	Belinda	41	+1-922-278-8375x211
110101196407104603	Anthony	54	001-461-627-5292
110101196803223943	Becky	41	920.360.5914x489
110101196909127547	Scott	37	001-153-688-6315
110101197011228296	Alexander	43	011.380.8012x255
110101197204248910	Christopher	53	+1-242-372-4582
110101197212145297	Jonathan	54	001-403-982-1744x8060
110101197401176503	David	28	043.438.0606
110101197404029144	Cynthia	65	8588019060
110101197507058250	Erin	42	267.199.0167x98313
110101197607234259	Cynthia	49	001-136-356-1089

(a) 查询 all 乘客信息

Query OK, 0 rows affected (0.00 sec)

order_id	id_card	moneys	payment
1	110101199001011234	1250.50	1
5	110101197507058250	2026.66	1
6	110101198011160376	539.77	1
7	110101195011276882	785.78	1
8	110101196009127547	1309.29	1
9	110101198610269070	2773.37	1
10	110101195005186187	711.23	1
11	110101198709081849	1567.04	1
14	110101199022114217	2019.16	1
15	110101198909150403	998.95	1
16	110101199066929067	1221.00	1
17	110101195710268104	2095.06	1
18	110101196205083417	916.85	1
19	110101198908252802	2199.78	1
21	11010119907095328	2097.73	1
24	110101195010215457	2662.74	1
25	110101199407102760	1268.23	1
26	110101197404029144	1369.64	1
32	110101196007104603	804.88	1
33	110101198610102751	2111.50	1
34	110101196009127547	1110.90	1
36	110101196010073893	1305.71	1
37	110101198208177794	2011.74	1
38	110101197011228296	1044.27	1
39	110101195603061279	1945.53	1
41	110101197204248910	2055.58	1
44	110101195310276691	1333.07	1
48	110101195510198228	2737.01	1
49	110101197006101216	2454.11	1
51	110101197705159207	2171.31	1
52	110101196003093909	1924.43	1

31 rows in set (0.00 sec)

(b) 所有已支付的订单

Query OK, 0 rows affected (0.00 sec)

id_card	gate	boarding_time
110101199806092967	B28	2024-04-15 03:26:08
110101199611172474	C15	2024-04-15 23:55:33
110101199103086642	C13	2024-04-24 06:29:21
110101198306273248	A24	2024-04-22 21:02:35
110101198208177794	C12	2024-04-25 01:15:01
110101198908196586	B2	2024-04-24 13:25:27
110101195510198228	C13	2024-04-17 21:28:13
110101199407095228	A25	2024-04-23 10:59:58
110101198908196586	A22	2024-04-17 05:43:10
110101195010215457	A17	2024-04-12 13:45:30
110101197705159207	A3	2024-04-20 11:05:10

11 rows in set (0.00 sec)

(c) 查询 CA1234 票务信息

图 12: 单表查询

5.2 多表连接查询

```

1  -- 查询所有乘客及其所订的航班号、舱位信息
2  SELECT p.passname, t.flight_number, t.cabin_seat
3  FROM Passenger p, Ticket t
4  WHERE p.id_card=t.id_card;

5

6

7  -- 查询乘客姓名、订单金额
8  SELECT p.passname, o.moneys
9  FROM Passenger p, Orderinfo o
10 WHERE p.id_card = o.id_card;

11

12 -- 查询每个航班的售出票数，筛选出票数大于5的航班
13 SELECT t.flight_number, COUNT(*) as cnt
14 FROM Ticket t, Flight f
15 WHERE t.flight_number = f.flight_number
16 GROUP BY t.flight_number
17 HAVING COUNT(*)>5;

18

19 -- 查询订购了航班 CA1234 的所有乘客的姓名
20 SELECT passname
21 FROM Passenger
22 WHERE id_card IN (
23     SELECT id_card
24     FROM Ticket
25     WHERE flight_number = 'CA1234'
26 );
27

```

```

28 -- 找出没有订过机票的乘客所有信息
29 SELECT *
30 FROM Passenger p
31 WHERE NOT EXISTS(
32     SELECT *
33     FROM Ticket t
34     WHERE t.id_card = p.id_card
35 );

```

查询的结果如图13所示。

Query OK, 0 rows affected (0.00 sec)

passname	flight_number	cabin_seat
Robin	MU4241	Y
Kimberly	MU4241	Y
Matthew	CA7836	C
Candice	CA1112	C
Candice	MU7290	Y
Amber	MU7290	Y
Jillian	MU7290	C
Belinda	CA7836	C
Alexander	CA1112	C
Jonathon	CA1112	C
Erin	CA7836	C
Eric	CA2849	C
Tara	CA1112	C
Tara	MU4241	Y
Michael	CA7836	Y
Monica	MU7290	Y
Monica	MU4241	C
Earl	CA2849	Y
Earl	MU4241	Y
Mallory	CA1112	C
Jared	CA7836	C
Chris	CA2849	C
Joshua	CA7836	Y
Sarah	CA2849	Y
Sherry	MU4241	Y
Sherry	MU4241	C
Adam	MU7290	C

(a) 所有乘客及所订的航班号、舱位信息

Query OK, 0 rows affected (0.00 sec)

passname	moneys
Robin	2662.74
Rachel	785.78
Mary	851.51
Kimberly	2737.81
Sheryl	1333.07
Patrick	1945.53
Matthew	2595.06
Angelica	1549.22
Candice	1118.90
Amber	1305.71
Jillian	916.85
Belinda	1369.09
Anthony	504.88
Becky	1673.81
Scott	1309.29
Alexander	1414.27
Christopher	2955.58
Jonathon	892.12
David	2584.45

(b) 查询乘客姓名、订单金额

Query OK, 0 rows affected (0.00 sec)

flight_number	cnt
CA1112	13
CA7836	8
MU4241	11
MU7290	13

4 rows in set (0.00 sec)

(c) 查询每个航班的售出票数，筛选出票数大于 5 的航班

Query OK, 0 rows affected (0.00 sec)

passname
Zhangsan

1 row in set (0.00 sec)

(d) 查询订购了航班 CA1234 的所有乘客的姓名

Query OK, 0 rows affected (0.00 sec)

id_card	passname	age	phone
110101195011276882	Rachel	50	(695)853-1407x8976
110101195005286187	Mary	70	572.245.9752x801
110101195010276591	Sheryl	68	002.636.5506
110101195003661779	Patrick	40	(746)521-7109x5387
110101195002078356	Angelica	39	671.252.2284x988
110101196007109463	Anthony	54	001-461-627-5292
110101196003223943	Becky	41	928.360.5910x489
110101196009127547	Scott	37	001-153-608-6315
110101197204248910	Christopher	53	+1-242-372-4582
110101197001175503	David	28	843.438.0606
110101197000292104	Cynthia	65	8588019068
110101197007234259	Cynthia	49	001-136-356-1089
110101198411164376	Lindsey	39	(247)989-2111x256
110101198502083115	Cameron	67	881-408-4726x2532
110101198610299070	Shirley	50	(132)996-0514
110101199602114217	Hayley	44	309-454-1411x140
110101198006024863	Dorothy	52	815.165.9534x0806

17 rows in set (0.00 sec)

(e) 找出没有订过机票的乘客所有信息

图 13: 多表连接查询

6 题目 6

题目要求：请尝试练习在某些表上建立唯一索引和聚集索引的方法，并将建索引的语句写入建库脚本中

6.1 唯一索引

唯一索引确保某字段值唯一，防止重复数据。

```
1 -- 创建唯一索引：手机号不重复
2 CREATE UNIQUE INDEX id_passphone ON Passenger(phone);
3
4 -- 创建唯一索引：同一航班一个座位只能安排给一个人
5 CREATE UNIQUE INDEX id_ticseat ON Ticket(flight_number, seat_number);
```

输入语句到 Mysql 中，第一条语句创立成功，第二条语句出现报错（如图14）：ERROR 1062 (23000): Duplicate entry 'MU7290-20B' for key 'ticket.id_ticseat'。出现的原因是在 Ticket 表中，存在两条记录的 (flight_number, seat_number) 都是 MU7290、20B。因此在输入数据前，应当先行建立索引，否则造成数据重复，从而出错。

```
mysql> CREATE UNIQUE INDEX id_passphone ON Passenger(phone);
Query OK, 0 rows affected (0.13 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE UNIQUE INDEX id_ticseat ON Ticket(flight_number, seat_number);
ERROR 1062 (23000): Duplicate entry 'MU7290-20B' for key 'ticket.id_ticseat'
```

图 14: 唯一索引

6.2 聚集索引

聚集索引（也叫聚簇索引，Clustered Index）指的是索引项的顺序与表中记录的物理顺序一致的索引组织。

```
1 -- 创建聚集索引：将表中所有数据按 id_card 顺序排列和存储
2 CREATE CLUSTERED INDEX idx_passenger_id ON Passenger(id_card);
```

```
mysql> CREATE CLUSTERED INDEX idx_passenger_id ON Passenger(id_card);
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'CLUSTERED INDEX idx_passenger_id ON Passenger(id_card)' at line 1
```

图 15: 聚集索引语句报错

将上方语句输入后，出现报错（图15）。因此，为解决本问题，本人查询了 CSDN 文章与 MySQL 官方文档。

17.6.2.1 Clustered and Secondary Indexes

Each table has a special index called the clustered index that stores row data. Typically, the clustered index is synonymous with the primary key. To get the best performance from queries, inserts, and other database operations, it is important to understand how uses the clustered index to optimize the common lookup and DML operations. InnoDBInnoDB

- When you define a on a table, uses it as the clustered index. A primary key should be defined for each table. If there is no logical unique and non-null column or set of columns to use a the primary key, add an auto-increment column. Auto-increment column values are unique and are added automatically as new rows are inserted. PRIMARY KEYInnoDB
- If you do not define a for a table, uses the first index with all key columns defined as as the clustered index. PRIMARY KEYInnoDBUNIQUENOT NULL
- If a table has no or suitable index, generates a hidden clustered index named on a synthetic column that contains row ID values. The rows are ordered by the row ID that assigns. The row ID is a 6-byte field that increases monotonically as new rows are inserted. Thus, the rows ordered by the row ID are physically in order of insertion. PRIMARY KEYUNIQUEInnoDBGEN_CLUST_INDEXInnoDB

图 16: MySQL 官方文档阐述聚集索引

从官方文档（图16）与其他 CSDN 文章可知，在 MySQL（InnoDB）中，不能显式创建聚集索引。只要设置了主键，InnoDB 引擎就会自动使用它作为聚集索引，**表数据会按主键的顺序物理存储**。SQL Server 和部分 Oracle（使用 IOT）等数据库系统是支持这样子的语法，但是 MySQL 不支持。因此，刚刚的语句仅作示例，无法展现实际效果。

6.3 MySQL 中的 Clustered Index

在 **MySQL InnoDB** 存储引擎中，聚集索引（Clustered Index）决定了表中数据的物理存储顺序。每张 InnoDB 表都有且只有一个聚集索引，通常情况下就是该表的 **主键**。聚集索引的选择遵循以下规则：

1. 若定义了 **PRIMARY KEY**，该主键自动成为聚集索引。
2. 若未定义主键，但存在第一个 **UNIQUE 且 NOT NULL** 的索引字段，则该字段作为聚集索引。
3. 若未定义主键，且无唯一非空字段，InnoDB 自动创建一个隐藏聚集索引，使用内部 row ID 排序。

情况一：定义主键作为聚集索引 例如在 Passenger 表中，设定了 id_card 为主键。

```
1 CREATE Table Passenger (  
2     id_card VARCHAR(20) PRIMARY KEY,  
3     passname VARCHAR(50) NOT NULL,  
4     age INT NOT NULL CHECK (  
5         age >= 0  
6         AND age < 120  
7     ),  
8     phone VARCHAR(100)  
9 );
```

情况二：无主键，使用唯一非空字段作为聚集索引 例如在 Airport 表中，假设没有主键，可以设置唯一非空字段作为聚集索引。

```
1 CREATE TABLE Airport (  
2     airport_code VARCHAR(10) NOT NULL UNIQUE,  
3     airport_name VARCHAR(100)  
4 );
```

虽然没有 PRIMARY KEY，但 airport_code 是 UNIQUE NOT NULL，因此它会自动作为聚集索引。

情况三：无主键或唯一约束，系统自动生成聚集索引 以下面的 TempLog 范例表为例：

```
1 CREATE TABLE TempLog (  
2     content TEXT,  
3     created_at DATETIME  
4 );
```

这个 TempLog 范例表中没有主键、也没有唯一约束字段。InnoDB 会自动生成一个隐式的聚集索引，叫 GEN_CLUST_INDEX，并用一个系统生成的 row ID（6 字节）对行排序。

7 题目 7

题目要求：若某个表中涉及百万甚至千万级以上的数据，请提出仿真这些数据方案，并在实验报告加以叙述。

在实际的数据库应用中，航空系统的业务场景某些核心表（如 Ticket、Orderinfo、Passenger）常常会涉及百万甚至千万级以上的数据量。根据作业要求，可以使用以下两种进行处理。

7.1 Python + Faker 库

由于在大一大创《面向需求响应的高铁快捷货物运输“车-货”匹配平台》曾使用 Python 语句生成仿真数据，因此知晓可以运用使用 Python 批量生成仿真数据。此处运用 Python，使用 faker、random 和 datetime 库，批量生成仿真数据并输出为 .sql 文件文件，结构和字段与原始表一致。Python 生成语句如下：

```
1 from faker import Faker  
2 import random  
3 from datetime import datetime, timedelta  
4  
5 faker = Faker(locale='en_US')  
6 random.seed(42)  
7  
8 # 参数设置
```

```

9 num_passengers = 100000
10 tickets_per_passenger = 2
11
12 # ===== 生成机场列表 (共20个) =====
13 airport_codes = set()
14 while len(airport_codes) < 20:
15     airport_codes.add(faker.unique.lexify(text='???').upper())
16 airport_list = [
17     (code, f"{faker.city()}_International_Airport",
18     faker.city(), random.choice(['3C', '4D', '4F', '4C']))
19     for code in airport_codes
20 ]
21
22 # ===== 生成航空公司 (共20个) =====
23 airline_codes = set()
24 while len(airline_codes) < 20:
25     airline_codes.add(faker.unique.lexify(text='??').upper())
26 airline_list = [
27     (code, f"{faker.company()}_Airlines", faker.phone_number())
28     for code in airline_codes
29 ]
30
31 # ===== 生成航班 (共100个) =====
32 flight_list = []
33 used_flight_numbers = set()
34 for _ in range(100):
35     airline = random.choice(airline_list)
36     while True:
37         flight_number = f"{airline[0]}{random.randint(1000, 9999)}"
38         if flight_number not in used_flight_numbers:
39             used_flight_numbers.add(flight_number)
40             break
41     model = random.choice(
42         ["Boeing737", "AirbusA320", "AirbusA321", "Boeing787", "
43         AirbusA319"])
44     departure, arrival = random.sample(airport_list, 2)
45     flight_list.append(
46         (flight_number, model, departure[0], arrival[0], airline[0]))
47
48 # ===== 写入 SQL 文件 =====
49 with open("exp3_data.sql", "w", encoding="utf-8") as f:

```

```

49     # Airport
50     f.write("--\nINSERT\nINTO\nAirport\n")
51     for a in airport_list:
52         f.write(
53             f"INSERT\nINTO\nAirport\nVALUES\n('{a[0]}',\n'{a[1]}',\n'{a[2]}',\n'{a[3]}')\n);\n")
54
55     # Airline
56     f.write("\n--\nINSERT\nINTO\nAirline\n")
57     for a in airline_list:
58         f.write(
59             f"INSERT\nINTO\nAirline\nVALUES\n('{a[0]}',\n'{a[1]}',\n'{a[2]}')\n);\n")
60
61     # Flight
62     f.write("\n--\nINSERT\nINTO\nFlight\n")
63     for fli in flight_list:
64         dep_time = datetime(2024, 4, 10, 8, 0)
65         arr_time = dep_time + timedelta(hours=2, minutes=30)
66         f.write(f"INSERT\nINTO\nFlight\nVALUES\n('{fli[0]}',\n'{fli[1]}',\n
67             '{dep_time.strftime('%Y-%m-%d\n%H:%M:%S')}',\n
68             f" '{arr_time.strftime('%Y-%m-%d\n%H:%M:%S')}',\n'{fli[2]}',\n'{fli[3]}',\n'{fli[4]}')\n);\n")
69
70     # Passenger, Ticket, Orderinfo
71     f.write("\n--\nINSERT\nINTO\nPassenger,\nTicket,\nOrderinfo\n")
72     for i in range(num_passengers):
73         id_card = f"ID{10000000+i}"
74         name = faker.first_name()
75         age = random.randint(18, 65)
76         phone = faker.phone_number()
77         f.write(
78             f"INSERT\nINTO\nPassenger\nVALUES\n('{id_card}',\n'{name}',\n{age},\n'{phone}')\n);\n")
79
80     for _ in range(tickets_per_passenger):
81         flight = random.choice(flight_list)[0]
82         seat = random.choice(["12A", "14C", "15B", "19D", "20E"])
83         gate = random.choice(["A01", "B05", "C12", "D03"])
84         boarding_time = faker.date_time_between(
            start_date=datetime(2024, 4, 10, 6, 0),

```

```

85         end_date=datetime(2024, 4, 10, 9, 0)
86     ).strftime('%Y-%m-%d_%H:%M:%S')
87     cabin = random.choice(["Y", "C"])
88     f.write(f"INSERT INTO Ticket (flight_number, id_card, seat_number, gate, boarding_time, cabin_seat) "
89            f"VALUES ('{flight}', '{id_card}', '{seat}', '{gate}', '{boarding_time}', '{cabin}');\n")
90
91     amount = round(random.uniform(300.0, 1500.0), 2)
92     payment = random.choice([0, 1])
93     f.write(
94         f"INSERT INTO Orderinfo (id_card, moneys, payment) "
95         f"VALUES ('{id_card}', {amount}, {payment});\n")
96
97 print("SQL generation complete: exp3_data.sql")

```

通过 python 生成 sql 文件后，进行数据插入（如图17）。本次仿真大约用 65 分钟。



```

MySQL 9.0 Command Line Cli  x  +  v
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.00 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)

```

图 17: Python + Faker 库

7.2 MySQL 存储过程批量插入

通过 WHILE 循环结构在数据库内快速插入大量测试数据，例如用于生成连续身份证号、自动编号的座位号等。这里运用 GPT，生成代码进行测试。

```

1 DELIMITER //

```

```

2 CREATE PROCEDURE insert_test_data(IN total_rows INT)
3 BEGIN
4     DECLARE i INT DEFAULT 1;
5     DECLARE id_prefix VARCHAR(10) DEFAULT 'ID';
6     DECLARE base_card BIGINT DEFAULT 10000000;
7     DECLARE seat_list JSON;
8     SET seat_list = JSON_ARRAY("12A", "14C", "15B", "19D", "20E");
9
10    WHILE i <= total_rows DO
11        -- 构造字段
12        SET @id_card = CONCAT(id_prefix, base_card + i);
13        SET @name = CONCAT('User', i);
14        SET @age = FLOOR(RAND() * 50) + 18;
15        SET @phone = CONCAT('13', LPAD(FLOOR(RAND()*100000000), 8, '0'
16            '));
17
18        -- 插入乘客
19        INSERT INTO Passenger (id_card, passname, age, phone)
20        VALUES (@id_card, @name, @age, @phone);
21
22        -- 随机航班、座位号、登机口、舱位
23        SET @flight_number = IF(RAND() > 0.5, 'CA1234', 'MU5678');
24        SET @seat = JSON_UNQUOTE(JSON_EXTRACT(seat_list, CONCAT('$[',
25            FLOOR(RAND()*5), ']')));
26        SET @gate = CONCAT(CHAR(FLOOR(RAND()*4)+65), LPAD(FLOOR(RAND
27            ()*10), 2, '0'));
28        SET @boarding_time = NOW();
29
30        -- 插入机票
31        INSERT INTO Ticket (flight_number, id_card, seat_number, gate
32            , boarding_time, cabin_seat)
33        VALUES (@flight_number, @id_card, @seat, @gate,
34            @boarding_time, IF(RAND() > 0.7, 'C', 'Y'));
35
36        -- 插入订单
37        INSERT INTO Orderinfo (id_card, moneys, payment)
38        VALUES (@id_card, ROUND(RAND() * 1000 + 500, 2), FLOOR(RAND()
39            *2));
40
41        SET i = i + 1;
42    END WHILE;

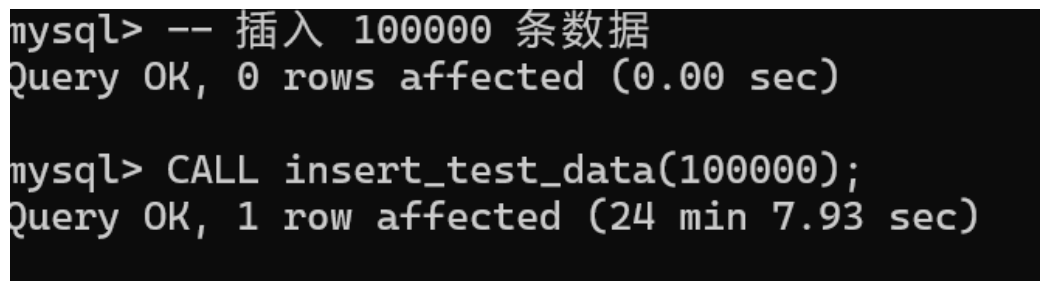
```

```

37 END //
38 DELIMITER ;
39
40 -- 插入 100000 条数据
41 CALL insert_test_data(100000);

```

通过 MySQL 插入数据后，进行数据插入（如图18）。本次仿真大约用时 24 分钟。



```

mysql> -- 插入 100000 条数据
Query OK, 0 rows affected (0.00 sec)

mysql> CALL insert_test_data(100000);
Query OK, 1 row affected (24 min 7.93 sec)

```

图 18: MySQL 存储过程批量插入

8 题目 8

8.1 问题 4

题目要求：用 *SQL* 语句建立第 2 章习题 2 第 6 题 *SPJ* 数据库的 4 个表 (*S*、*P*、*J* 和 *SPJ*)，并针对建立的 4 个表用 *SQL* 语句完成该习题中的查询。

语句如下：

```

1 CREATE DATABASE homework;
2 USE homework;
3
4 -- S表（供应商表）
5 CREATE TABLE S (
6     SNO CHAR(2) PRIMARY KEY,
7     SNAME VARCHAR(20),
8     S_STATUS INT,
9     CITY VARCHAR(20)
10 );
11
12 -- P表（零件表）
13 CREATE TABLE P (
14     PNO CHAR(2) PRIMARY KEY,
15     PNAME VARCHAR(20),
16     COLOR VARCHAR(10),
17     P_WEIGHT INT
18 );
19

```



```

20 -- J表 (工程项目表)
21 CREATE TABLE J (
22     JNO CHAR(2) PRIMARY KEY,
23     JNAME VARCHAR(20),
24     CITY VARCHAR(20)
25 );
26
27 -- SPJ表 (供应情况表)
28 CREATE TABLE SPJ (
29     SNO CHAR(2),
30     PNO CHAR(2),
31     JNO CHAR(2),
32     QTY INT,
33     PRIMARY KEY (SNO, PNO, JNO),
34     FOREIGN KEY (SNO) REFERENCES S(SNO),
35     FOREIGN KEY (PNO) REFERENCES P(PNO),
36     FOREIGN KEY (JNO) REFERENCES J(JNO)
37 );
38
39 -- S表数据
40 INSERT INTO S (SNO, SNAME, S_STATUS, CITY) VALUES
41     ('S1', '精益', 20, '天津'),
42     ('S2', '盛锡', 10, '北京'),
43     ('S3', '东方红', 30, '北京'),
44     ('S4', '丰泰盛', 20, '天津'),
45     ('S5', '为民', 30, '上海');
46
47 -- P表数据
48 INSERT INTO P (PNO, PNAME, COLOR, P_WEIGHT) VALUES
49     ('P1', '螺母', '红', 12),
50     ('P2', '螺栓', '绿', 17),
51     ('P3', '螺丝刀', '蓝', 14),
52     ('P4', '螺丝刀', '红', 14),
53     ('P5', '凸轮', '蓝', 40),
54     ('P6', '齿轮', '红', 30);
55
56 -- J表数据
57 INSERT INTO J (JNO, JNAME, CITY) VALUES
58     ('J1', '三建', '北京'),
59     ('J2', '一汽', '长春'),
60     ('J3', '弹簧厂', '天津'),

```

```

61 ('J4', '造船厂', '天津'),
62 ('J5', '机车厂', '唐山'),
63 ('J6', '无线电厂', '常州'),
64 ('J7', '半导体厂', '南京');
65
66 -- SPJ表数据
67 INSERT INTO SPJ (SNO, PNO, JNO, QTY) VALUES
68 ('S1', 'P1', 'J1', 200),
69 ('S1', 'P1', 'J3', 100),
70 ('S1', 'P1', 'J4', 700),
71 ('S1', 'P2', 'J2', 100),
72 ('S2', 'P3', 'J1', 400),
73 ('S2', 'P3', 'J2', 200),
74 ('S2', 'P3', 'J4', 500),
75 ('S2', 'P3', 'J5', 400),
76 ('S2', 'P5', 'J1', 400),
77 ('S2', 'P5', 'J2', 100),
78 ('S3', 'P1', 'J1', 200),
79 ('S3', 'P3', 'J1', 200),
80 ('S4', 'P5', 'J1', 100),
81 ('S4', 'P6', 'J3', 300),
82 ('S4', 'P6', 'J4', 200),
83 ('S5', 'P2', 'J4', 100),
84 ('S5', 'P3', 'J1', 200),
85 ('S5', 'P6', 'J2', 200),
86 ('S5', 'P6', 'J4', 500);

```

之后回答 5 个小问题:

1. 求供应工程 J1 零件的供应商代码 SNO

```

1  SELECT DISTINCT SNO
2  FROM SPJ
3  WHERE JNO = 'J1';

```

2. 求供应工程 J1 零件 P1 的供应商代码 SNO

```

1  SELECT DISTINCT SNO
2  FROM SPJ
3  WHERE JNO = 'J1' AND PNO = 'P1';

```

3. 求供应工程 J1 零件为红色的供应商代码 SNO

```

1  SELECT DISTINCT SNO

```

```

2      FROM SPJ
3      WHERE JNO = 'J1'
4      AND PNO IN(
5          SELECT PNO
6          FROM P
7          WHERE COLOR = '红'
8      );

```

4. 求没有使用天津供应商生产的红色零件的工程号 JNO

```

1      SELECT DISTINCT JNO
2      FROM SPJ
3      WHERE JNO NOT IN (
4          SELECT JNO
5          FROM SPJ
6          WHERE SNO IN (
7              SELECT SNO
8              FROM S
9              WHERE CITY = '天津'
10         )
11         AND PNO IN (
12             SELECT PNO
13             FROM P
14             WHERE COLOR = '红'
15         )
16     );

```

5. 求至少使用了与供应商 S1 所供应的全部零件相同零件号的工程号 JNO

```

1      -- 方法1
2      SELECT DISTINCT SPJ1.JNO
3      FROM SPJ AS SPJ1
4      WHERE NOT EXISTS (
5          SELECT *
6          FROM SPJ AS SPJ2
7          WHERE SPJ2.SNO = 'S1'
8          AND NOT EXISTS (
9              SELECT *
10             FROM SPJ AS SPJ3
11             WHERE SPJ3.JNO = SPJ1.JNO
12                 AND SPJ3.PNO = SPJ2.PNO
13         )

```

```

14     );
15
16     -- 方法2
17     SELECT DISTINCT JNO
18     FROM SPJ AS SPJ1
19     WHERE NOT EXISTS (
20         SELECT PNO
21         FROM SPJ
22         WHERE SNO = 'S1'
23         EXCEPT
24         SELECT PNO
25         FROM SPJ AS SPJ2
26         WHERE SPJ2.JNO = SPJ1.JNO
27     );

```

8.2 问题 5

题目要求：针对第 4 题中的 4 个表 *S*、*P*、*J* 及 *SPJ*，试用 *SQL* 完成以下各项操作：

1. 找出所有供应商的姓名和所在城市。
2. 找出所有零件的名称、颜色、重量。
3. 找出使用供应商 *S1* 所供应零件的工程代码。
4. 找出工程项目 *J2* 使用的各种零件的名称及其数量。
5. 找出上海厂商供应的所有零件代码。
6. 找出使用上海产的零件的工程名称。
7. 找出没有使用天津产的零件的工程代码。
8. 把全部红色零件的颜色改成蓝色。
9. 把由 *S5* 供给 *J2* 的零件 *P6* 改为由 *S3* 供应。
10. 从供应商关系中删除 *S2* 的记录，并从供应情况关系中删除相应的记录。
11. 请将 (*S2*,*J6*,*P4*,200) 插入供应情况关系。

1. 找出所有供应商的姓名和所在城市

```

1     SELECT SNAME, CITY
2     FROM S;

```

2. 找出所有零件的名称、颜色、重量

```
1      SELECT PNAME, COLOR, P_WEIGHT
2      FROM P;
```

3. 找出使用供应商 S1 所供应零件的工程代码

```
1      SELECT DISTINCT JNO
2      FROM SPJ
3      WHERE SNO = 'S1';
```

4. 求供应工程 J1 零件的供应商代码 SNO

```
1      SELECT P.PNAME, SPJ.QTY
2      FROM P.SPJ
3      WHERE P.PNO = SPJ.PNO
4             AND SPJ.JNO = 'J2';
```

5. 找出上海厂商供应的所有零件代码

```
1      SELECT DISTINCT PNO
2      FROM SPJ
3      WHERE SNO IN (
4          SELECT SNO
5          FROM S
6          WHERE CITY = '上海'
7      );
```

6. 找出使用上海产的零件的工程名称

```
1      SELECT DISTINCT JNAME
2      FROM J
3      WHERE JNO IN (
4          SELECT JNO
5          FROM SPJ
6          WHERE SNO IN (
7              SELECT SNO
8              FROM S
9              WHERE CITY = '上海'
10         )
11     );
```

7. 找出没有使用天津产的零件的工程代码

```

1      SELECT DISTINCT JNO
2      FROM J
3      WHERE NOT EXISTS (
4          SELECT *
5          FROM SPJ
6          WHERE SPJ.JNO = J.JNO
7              AND SPJ.SNO IN (
8                  SELECT SNO
9                  FROM S
10                 WHERE CITY = '天津'
11             )
12      );

```

8. 把全部红色零件的颜色改成蓝色

```

1      UPDATE P
2      SET COLOR = '蓝'
3      WHERE COLOR = '红';

```

9. 把由 S5 供给 J2 的零件 P6 改为由 S3 供应

```

1      UPDATE SPJ
2      SET SNO = 'S3'
3      WHERE SNO = 'S5'
4          AND JNO = 'J4'
5          AND PNO = 'P6';

```

10. 从供应商关系中删除 S2 的记录，并从供应情况关系中删除相应的记录。

```

1      DELETE FROM SPJ
2      WHERE SNO = 'S2';
3      DELETE FROM S
4      WHERE SNO = 'S2';

```

11. 插入 (S2, J6, P4, 200) 到供应情况表

```

1      INSERT INTO SPJ (SNO, JNO, PNO, QTY)
2      VALUES ('S2', 'J6', 'P4', 200);

```

8.3 问题 9

题目要求：针对第 4 题中的 4 个表，为三建工程项目建立一个供应情况的视图，包括供应商代码 (SNO)、零件代码 (PNO)、供应数量 (QTY)。针对该视图完成下列查询：

1. 找出三建工程项目使用的各种零件代码及其数量。
2. 找出供应商 S1 供应三建工程情况。

创建视图三建工程项目的供应情况如下：

```
1 SELECT SNO, PNO, QTY
2 FROM SPJ
3 WHERE
4     JNO = (
5         SELECT JNO
6         FROM J
7         WHERE
8             JNAME = '三建'
9     );
```

1. 找出三建工程项目使用的各种零件代码及其数量

```
1     SELECT PNO, QTY
2     FROM V_SPJ;
```

2. 找出供应商 S1 供应三建工程情况

```
1     SELECT *
2     FROM V_SPJ
3     WHERE SNO = 'S1';
```